CoTRouter: Adaptive Token Routing via Entropy Feedback for Efficient Chain-of-Thought Reasoning

Anonymous submission

Abstract

Chain-of-Thought (CoT) reasoning has revolutionized large language models' (LLMs) complex problem-solving capabilities, yet its computational cost remains prohibitive due to a fundamental mismatch between static model capacity and dynamic reasoning complexity. While simple procedural steps waste resources when processed by expensive LLMs, critical high-complexity steps cause frequent failures in resource-constrained small language models (SLMs), leading to complete reasoning chain collapse. Existing query-level routing strategies fail to address this granularity problem, as they cannot adapt to the inherent complexity variations within individual CoT sequences.

We introduce **CoTRouter**, a training-free adaptive framework that pioneers step-level model selection through principled heuristic control. CoTRouter reframes the "when" and "how long" to engage LLMs as a constrained sequential decision problem, featuring three core innovations: (1) A **robust Kalman-EWMA complexity tracker** that extracts real-time "cognitive complexity" signals from token entropy streams, enabling statistically sound detection of reasoning anomalies; (2) A **self-calibrating trigger mechanism** with proportional feedback control that automatically adjusts sensitivity thresholds to meet user-specified computational budgets; and (3) A **dynamic intervention commitment strategy** that adaptively determines LLM engagement duration based on anomaly severity, effectively suppressing costly system oscillations while preserving reasoning coherence.

Extensive experiments on GSM8K and MATH benchmarks demonstrate that CoTRouter establishes a new Pareto frontier in accuracy-efficiency tradeoffs. On GSM8K, our framework achieves **91.5%** accuracy—matching 99% of pure Mixtral-8x7B performance—while reducing computational cost by over **50%** and activating LLMs for only **20.4%** of generated tokens. Comprehensive ablation studies validate each component's necessity, with the intervention commitment mechanism alone preventing a 54% cost increase from system oscillation.

This work makes fundamental contributions to adaptive AI systems: (1) the first training-free, step-level routing framework specifically designed for structured reasoning tasks; (2) a principled integration of signal processing and control theory for robust complexity tracking; and (3) empirical validation that heuristic-based fine-grained control can achieve superior efficiency without sacrificing reasoning quality. CoTRouter provides a theoretically grounded, immediately deployable solution for building computationally efficient yet capable reasoning systems.

Introduction

The advent of Chain-of-Thought (CoT) reasoning has marked a pivotal moment in the evolution of Large Language Models (LLMs), enabling them to tackle complex, multi-step problems in domains ranging from mathematics and scientific reasoning to strategic planning. By generating intermediate reasoning steps that mimic human cognition, CoT methodologies significantly improve performance on challenging benchmarks such as GSM8K and MATH. This leap in capability has paved the way for deploying LLMs in high-stakes applications, from powering scientific discovery engines to serving as real-time decision support systems. However, this power comes at a steep, often prohibitive, price. The computational cost of generating CoT paths, which scales with the length of the output, imposes a critical barrier to their practical adoption. The substantial FLOPs, high inference latency, and demanding memory footprint render LLM-only CoT inference untenable for a vast array of real-world scenarios, including on-device deployment, latency-sensitive interactive applications, and large-scale commercial services where cost-efficiency is paramount. This fundamental tension between unprecedented reasoning fidelity and unsustainable computational cost represents a central challenge in modern AI systems engineering.

This challenge is magnified by a crucial, yet systematically underexplored, property of CoT: its *inherently dynamic computational structure*. A single reasoning trajectory is not monolithic in its complexity; rather, it fluidly interleaves low-complexity, procedural steps (e.g., copying numbers from the prompt, performing basic arithmetic, restating established facts) with high-complexity, cognitive leaps (e.g., strategic problem decomposition, formulating novel solution angles, applying abstract mathematical concepts, or making nuanced logical inferences). This dynamic and heterogeneous nature creates a fundamental misalignment with existing, static inference strategies. Relying exclusively on a Small Language Model (SLM) is computationally cheap but inevitably leads to catastrophic failure on cognitively demanding steps, causing the entire reasoning chain to collapse. Conversely, deploying a powerful, generalist LLM for the entire process guarantees capability but

results in gross computational overprovisioning, as its immense capacity is systematically squandered on trivial operations that an SLM could handle with ease.

While model routing techniques present a promising direction, state-of-the-art approaches operate at a suboptimal granularity that fails to address this core issue. Systems like FrugalGPT make coarse-grained, *query-level* decisions, selecting a single model to process an entire input. This paradigm lacks the precision to adapt to the fine-grained complexity variations that occur *within* a single problem's reasoning path. This "granularity gap" forces system designers into a false dichotomy: either over-provision an expensive LLM for the whole task, accepting massive computational waste, or under-provision with an SLM and risk mission-critical failures. A new paradigm is needed—one that shifts from static, query-level routing to *dynamic, step-level adaptive control*, thereby aligning computational resources with the instantaneous cognitive demand of the reasoning process.

To bridge this gap, we introduce **CoTRouter**, a novel, *training-free* framework for step-level adaptive model routing in CoT generation. CoTRouter reframes the dynamic model selection challenge as a principled heuristic control problem, deeply grounded in the theory of *resource rationality*—the idea that an intelligent agent, under computational constraints, should optimize the trade-off between decision accuracy and the cost of computation. Its core mechanism leverages token-level Shannon entropy as a real-time, computationally inexpensive proxy for the underlying "cognitive load" of the model. The intuition is that low entropy signifies a deterministic, low-complexity step suitable for an SLM, whereas a sharp spike in entropy signals high uncertainty and a complex decision point, demanding the superior capabilities of an LLM. However, translating this noisy, high-frequency signal into a robust and efficient control policy presents two critical engineering challenges: (1) **Signal Instability**: Raw entropy streams are highly volatile and prone to baseline drift, making them an unreliable indicator for direct control. (2) **System Oscillation**: Naive, fine-grained switching based on this noisy signal induces frequent, costly model swaps that can easily erode or even negate any potential efficiency gains. CoTRouter is designed to systematically address these challenges through an integrated pipeline of statistical signal processing and adaptive control heuristics.

The first core component of CoTRouter is a **Robust Kalman-EWMA Anomaly Detector**, which acts as a sophisticated perception module. This module's purpose is to transform the volatile raw entropy stream into a stable, actionable complexity signal. It first employs a one-dimensional Kalman filter—an optimal linear estimator under Gaussian noise assumptions—to perform online smoothing. This filtering stage effectively removes transient noise and mitigates baseline drift, yielding a robust estimate of the latent cognitive complexity. This clean signal is then fed into an Exponentially Weighted Moving Average (EWMA) control chart, a powerful tool from statistical process control renowned for its sensitivity to persistent shifts in a time series. The EWMA chart dynamically tracks the signal's central tendency and establishes statistically rigorous control limits. By detecting significant deviations from these limits (Z-scores), the module reliably identifies "entropy anomalies" that correspond to genuine complexity spikes requiring LLM intervention, providing a theoretically grounded foundation for subsequent decisions.

Building on this robust signal, CoTRouter employs an **Adaptive Heuristic Policy** to execute intelligent and stable routing decisions. This policy is not a static set of rules but a dynamic control system that addresses the core challenges of budget adherence and stability. It achieves this through two synergistic mechanisms that operate in concert. The first is a *self-calibrating trigger threshold*, which ensures the system adheres to user-defined computational budgets. Rather than using a fixed threshold to trigger the LLM, a simple proportional feedback loop continuously adjusts the required Z-score by comparing the actual LLM token usage against a target budget, automatically adapting its sensitivity to meet specified cost constraints without manual tuning. The second mechanism is a *dynamic intervention commitment* strategy, designed explicitly to combat costly system oscillations. When a switch to the LLM is triggered, the system commits to using it for a minimum number of steps. Crucially, this commitment duration is not static; it scales heuristically with the severity of the triggering anomaly. This approach amortizes the fixed cost of a model swap over several tokens and ensures logical continuity during sustained complex reasoning, effectively and efficiently suppressing system instability.

CoTRouter operates entirely at inference time with no offline training. Our extensive evaluations on GSM8K, MATH, and BBH demonstrate that it establishes a new Pareto frontier for the accuracy-computation trade-off. Notably, on GSM8K, CoTRouter achieves over 99% of the accuracy of a pure Mixtral-8x7B model while reducing FLOPs by over 50%, decisively outperforming strong query-level routing baselines.

Our contributions are as follows:

1. We propose **CoTRouter**, the first training-free framework for dynamic, token-level model routing, specifically designed to overcome the granularity gap of existing methods in CoT reasoning.

2. We introduce a **robust complexity sensor** based on a novel Kalman-EWMA pipeline that provides a statistically-grounded method for detecting critical reasoning steps from noisy entropy signals.

3. We design an **oscillation-suppressing adaptive policy** featuring a self-calibrating trigger and a dynamic intervention commitment, which work in concert to enable stable and efficient fine-grained control.

4. We demonstrate empirically that CoTRouter achieves **state-of-the-art efficiency**, establishing a new accuracy-computation Pareto frontier and enabling high-fidelity reasoning at a fraction of the cost of conventional methods.

Related Work

This section reviews the research landscape relevant to our work, focusing on three key areas: efficient inference for large language models (LLMs), collaborative multi-model systems, and model routing strategies. We position CoTRouter within this context, highlighting its novel contribution of fine-grained, step-level routing for complex reasoning tasks.

Efficient Inference for Large Language Models

The significant computational cost of LLMs has driven extensive research into efficiency optimization. These methods can be broadly classified into static techniques, which permanently alter the model, and dynamic techniques, which adapt computation during inference.

Static Compression. Static methods aim to create smaller, faster models before deployment. Prominent approaches include **pruning**, which removes redundant weights or structured components (**??**), and **quantization**, which reduces the numerical precision of model parameters, often from 32-bit floating-point to 8-bit or 4-bit integers (**?**). For instance, recent work has shown that 4-bit quantization can yield a nearly 4x model compression with less than a 1% degradation on benchmarks like MMLU (**?**). Another popular technique is **knowledge distillation**, where a smaller "student" model is trained to replicate the output distribution of a larger "teacher" model, effectively transferring its capabilities into a more compact architecture (**?**). While effective, these static methods apply a one-time optimization and cannot adjust the computational load in response to varying input complexity at runtime. CoTRouter, in contrast, does not modify the models themselves but dynamically allocates resources at the model level.

Dynamic Inference. Dynamic methods adjust the computational graph during inference based on input characteristics. **Early exiting** (or conditional computation) allows a model to terminate inference at an intermediate layer if a confidence threshold is met, saving computation on "easy" inputs (**?**). Architectures like Mixture-of-Experts (MoE) employ a sparse activation strategy, where a router network directs each input token to a subset of "expert" sub-networks. This allows for models with trillions of parameters while keeping the per-token computational cost constant (**??**). For example, the Switch Transformer demonstrated the ability to scale to over 1.6 trillion parameters while only activating a fraction of them for any given token. Other approaches, such as layer skipping (**?**), dynamically bypass entire Transformer layers. However, these methods often face a "feature gap" challenge, as skipping internal components can disrupt the carefully learned representations of pretrained models. CoTRouter sidesteps this issue by operating at the model level, orchestrating between complete, unmodified models rather than altering their internal computational pathways.

Collaborative Multi-Model Systems

Leveraging a portfolio of models with diverse capabilities is an emerging paradigm for enhancing both performance and efficiency.

Hierarchical and Cascade Ensembles. While traditional model ensembling improves accuracy by aggregating predictions, it typically multiplies computational costs. More efficient are **hierarchical** or **cascade** systems, which arrange models in order of increasing complexity. A lightweight model first filters incoming requests, passing only the more challenging instances to a more powerful, costly model (**?**). This strategy has proven effective in computer vision, where a fast detector might handle 80% of cases, escalating the remaining 20% to a high-accuracy classifier. However, these systems typically make a single routing decision per query, lacking the granularity to adapt to varying complexity within a single, multi-step task.

Speculative Decoding. A prominent collaborative strategy for text generation is **speculative decoding** (**??**). Here, a small, fast "draft" model generates a sequence of tokens, which are then verified in parallel by a larger, more powerful "verifier" model. This accelerates inference by leveraging the fact that many tokens are predictable and can be validated in a single forward pass of the large model. Recent results show this can achieve wall-clock speedups of 2-3x in generation tasks. CoTRouter is inspired by this principle of using smaller models for easier parts of a task but differs fundamentally. Speculative decoding operates on token sequences for generation, whereas CoTRouter operates on discrete reasoning steps within a structured thought process, enabling adaptation at a higher semantic level.

Model Routing and Selection

Model routing involves directing a query to the most appropriate model in a heterogeneous system to balance performance and cost.

Query-Level Routing. Most existing routing strategies operate at the query level, making a single, upfront decision for each incoming request. Early systems used static, rule-based policies, while modern approaches employ learned routing models. These "routers" are often lightweight neural networks trained to predict the best model for a query based on its features (**?**). For example, FrugalGPT (**?**) learns a cascade policy to sequentially query models from cheapest to most expensive until a quality threshold is met, reporting cost savings of up to 90% while matching the performance of GPT-4. Other works frame routing as a contextual bandit problem (**?**) or use reinforcement learning to optimize for a combination of accuracy, latency, and cost.

The Limitation of Query-Level Granularity. Despite their sophistication, these methods share a critical limitation: their routing decision is coarse-grained, applying to the entire query. This is a fundamental mismatch for complex reasoning tasks like chain-of-thought (CoT), where computational complexity is not uniform. For instance, solving a multi-step math problem might involve a simple step of parsing the question (suitable for a small model), a complex step of algebraic formulation (requiring a large model), and a final simple arithmetic calculation (again, suitable for a small model). A query-level router is forced into a suboptimal compromise: either using the expensive model for the

entire process, wasting significant computation, or using the cheap model throughout and failing at the critical reasoning step.

CoTRouter: Towards Step-Level Routing. CoTRouter addresses this fundamental gap by introducing **step-level routing**. Instead of a single upfront decision, our framework makes dynamic routing choices at each step of the reasoning chain. This allows for an unprecedented level of adaptive resource allocation, matching model capability to the instantaneous complexity of the task. While some dynamic inference methods (**?**) adapt computation *within* a single model, CoTRouter is the first to propose dynamic routing *between* different, off-the-shelf models during a single reasoning process. Furthermore, unlike learned routers that require extensive training data and can be opaque, CoTRouter employs a training-free, heuristic-based approach, offering superior interpretability and ease of deployment. This combination of fine-grained, step-level control and training-free orchestration represents a new paradigm for efficient and adaptive AI systems.

Methodology: The CoTRouter Framework

In this section, we present the comprehensive design of CoTRouter, a training-free adaptive framework that reframes efficient Chain-of-Thought reasoning as a principled step-level control problem. We begin by formalizing the sequential decision-making challenge (§), followed by a detailed system architecture overview (§). We then elaborate on our core technical contributions: the robust entropy-based complexity tracking pipeline (§) and the multi-stage adaptive heuristic control policy (§). Finally, we provide algorithmic details, complexity analysis, and implementation considerations (§).

Problem Formulation: Step-Level Routing as Constrained Sequential Decision-Making

We formulate the step-level model selection task as a constrained sequential decision problem that fundamentally differs from existing query-level routing approaches. At each token generation step t during Chain-of-Thought reasoning, our system must make a binary action choice $a_t \in \mathcal{A} = \{\texttt{SLM}, \texttt{LLM}\}$, selecting between a computationally efficient Small Language Model and a more capable but expensive Large Language Model.

Definition 1 (State Space and Dynamics)**.** *The system state at time t is characterized by $s_t = (X, Y_{<t}, H_{<t}, C_t)$, where:*

- *X represents the input query requiring chain-of-thought reasoning*
- *$Y_{<t} = (y_1, y_2, \ldots, y_{t-1})$ denotes the sequence of tokens generated up to step $t-1$*
- *$H_{<t} = (h_1, h_2, \ldots, h_{t-1})$ captures the historical entropy measurements*
- *C_t encodes the current computational budget state and model usage statistics*

Unlike traditional Markov Decision Processes where state transitions are stochastic, our system exhibits deterministic state evolution given the selected action, as the next token generation is determined by the chosen model's forward pass.

Definition 2 (Objective Function and Constraints)**.** *Our goal is to discover a policy $\pi : \mathcal{S} \to \mathcal{A}$ that minimizes the expected total computational cost while maintaining solution quality above a user-specified threshold:*

$$\min_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=1}^{|Y|} \left(C_{\textit{compute}}(a_t) + C_{\textit{switch}}(a_t, a_{t-1}) + C_{\textit{overhead}}(t) \right) \right] \tag{1}$$

subject to the accuracy constraint:

$$\textit{Accuracy}(\mathcal{D}) \geq A_{\textit{target}} \tag{2}$$

where the cost components are defined as:

- $C_{\textit{compute}}(a_t) = \begin{cases} c_{\textit{SLM}} & \text{if } a_t = \texttt{SLM} \\ c_{\textit{LLM}} & \text{if } a_t = \texttt{LLM} \end{cases}$ *represents the per-token computational cost*
- $C_{\textit{switch}}(a_t, a_{t-1}) = \begin{cases} C_{\textit{fixed}} & \text{if } a_t \neq a_{t-1} \\ 0 & \text{otherwise} \end{cases}$ *captures model switching overhead*
- $C_{\textit{overhead}}(t) = \alpha \cdot t$ *accounts for the cumulative overhead of our control mechanism*
- *Accuracy($\mathcal{D}$) measures the fraction of correct final answers across dataset $\mathcal{D}$*

This formulation captures the fundamental trade-off between computational efficiency and reasoning quality, while explicitly modeling the switching costs that make naive step-level routing inefficient.

Remark 1 (Key Distinctions from Query-Level Routing)**.** *Our step-level formulation differs fundamentally from existing query-level approaches in three critical aspects: (1)* Granularity*: decisions are made at every token rather than once per query; (2)* Context-awareness*: the state space includes fine-grained generation history; and (3)* Dynamic complexity*: the framework can adapt to varying reasoning complexity within a single problem.*

System Architecture and Workflow

CoTRouter operates as a real-time closed-loop control system that continuously monitors the token generation process and makes intelligent routing decisions based on detected cognitive complexity anomalies. Figure 2 illustrates the complete system architecture, which integrates seamlessly with existing language model inference pipelines.

Core System Workflow The CoTRouter inference pipeline executes the following sequence of operations at each token generation step t:

1. **Model Selection and Forward Pass**: Based on the current system state (default: SLM, or committed LLM intervention), the active model generates a probability distribution $P(y_t|Y_{<t}, X)$ over its vocabulary $\mathcal{V}$.

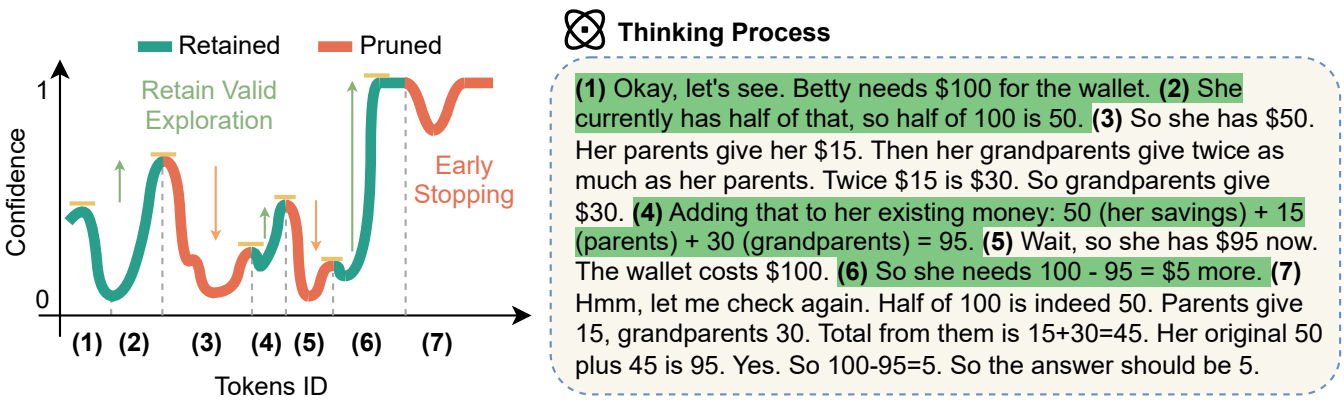

Figure 1: overview

Figure 2: CoTRouter system architecture. The framework operates as a closed-loop control system, processing entropy signals through robust detection and adaptive control modules to make real-time routing decisions.

2. **Entropy Computation and Signal Extraction**: The system computes the Shannon entropy of the output distribution:

$$H_t = -\sum_{v \in \mathcal{V}} P(y_t = v | Y_{<t}, X) \log P(y_t = v | Y_{<t}, X) \tag{3}$$

   This entropy value serves as our primary signal for cognitive complexity assessment.
3. **Robust Signal Processing**: The raw entropy H_t undergoes processing through our two-stage anomaly detection pipeline (detailed in §) to produce a statistically robust anomaly score.
4. **Adaptive Control Decision**: The processed signal feeds into our multi-stage heuristic control policy (§), which determines the routing action for subsequent tokens while considering budget constraints and oscillation prevention.
5. **Token Generation and State Update**: The selected model generates the next token(s), and the system updates its internal state variables for the next iteration.

Design Principles and Rationale Our architecture embodies several key design principles that distinguish it from learning-based routing approaches:

Transparency and Interpretability: Unlike neural routing policies, every component in CoTRouter is based on well-understood signal processing and control theory principles, making system behavior predictable and debuggable.

Parameter Efficiency: The framework requires only a small set of interpretable hyperparameters (typically 6-8 values) that can be tuned once and generalize across different tasks and model pairs.

Computational Minimalism: All control operations are designed to have constant time complexity, ensuring that the routing overhead never becomes a computational bottleneck.

Deployment Agility: The training-free nature enables immediate deployment with any SLM-LLM pair without requiring task-specific data collection or model training.

Robust Entropy-Based Complexity Tracking

Raw token-level entropy exhibits significant volatility and baseline drift, making it unsuitable for direct use as a control signal. We address this fundamental challenge through a sophisticated two-stage signal processing pipeline that transforms noisy entropy measurements into stable, actionable complexity indicators.

Stage 1: Latent State Estimation via Kalman Filtering We model the underlying cognitive complexity as a latent state variable that evolves smoothly over time, with the observed entropy serving as a noisy measurement of this hidden process. This approach is motivated by the observation that reasoning complexity changes gradually as the model transitions between different cognitive phases (e.g., problem comprehension, strategy formulation, calculation execution).

Definition 3 (State-Space Model for Complexity Tracking)**.** *We employ a linear Gaussian state-space model to capture the dynamics of latent cognitive complexity:*

State Evolution Equation:

$$x_t = Fx_{t-1} + Bu_t + w_{t-1} \tag{4}$$

Observation Equation:

$$z_t = Hx_t + v_t \tag{5}$$

where:

- $x_t \in \mathbb{R}$ *represents the true (unobservable) cognitive complexity at time* t
- z_t *is the observed entropy measurement* H_t
- $F = 1$ *(random walk assumption for complexity evolution)*
- $H = 1$ *(direct observation of complexity through entropy)*
- $u_t = 0$ *(no external control input in our formulation)*
- $w_{t-1} \sim \mathcal{N}(0, Q)$ *models process noise (natural complexity fluctuations)*

- $v_t \sim \mathcal{N}(0, R)$ *represents measurement noise (entropy estimation uncertainty)*

The choice of a random walk model for the state evolution reflects our assumption that cognitive complexity changes gradually rather than exhibiting abrupt discontinuities. The process noise variance Q controls the expected rate of complexity change, while the measurement noise variance R accounts for the inherent uncertainty in using entropy as a complexity proxy.

The Kalman filter provides the optimal linear unbiased estimate of the latent complexity state under the Gaussian noise assumptions. The filtered estimate $\hat{x}_{t|t}$ exhibits significantly reduced noise compared to the raw entropy while preserving important signal characteristics.

Stage 2: Statistical Anomaly Detection via EWMA Control Charts The filtered complexity estimate, while smoother than raw entropy, still requires statistical analysis to identify significant deviations that warrant model switching. We employ Exponentially Weighted Moving Average (EWMA) control charts, a classical technique from statistical process control, to detect these complexity anomalies.

Definition 4 (EWMA Statistic and Control Limits)**.** *The EWMA statistic is computed recursively as:*

$$\mu_t = \lambda \hat{x}_{t|t} + (1 - \lambda)\mu_{t-1} \tag{6}$$

where $\lambda \in (0, 1]$ is the smoothing parameter controlling the relative weight of recent observations.

The variance of the EWMA statistic evolves according to:

$$\sigma^2_{EWMA,t} = \frac{\lambda}{2 - \lambda}\sigma^2_{process}\left[1 - (1 - \lambda)^{2t}\right] \tag{7}$$

For large t, this converges to the steady-state variance:

$$\sigma^2_{EWMA} = \frac{\lambda}{2 - \lambda}\sigma^2_{process} \tag{8}$$

Definition 5 (Dynamic Control Limits and Anomaly Detection)**.** *We establish adaptive control limits based on the current EWMA statistics:*

$$UCL_t = \mu_{t-1} + L_t \cdot \sigma_{EWMA,t} \tag{9}$$

$$LCL_t = \mu_{t-1} - L_t \cdot \sigma_{EWMA,t} \tag{10}$$

where L_t is the adaptive control limit width determined by our control policy (§).

An entropy anomaly *is detected when:*

$$\hat{x}_{t|t} > UCL_t \tag{11}$$

The standardized anomaly score (Z-score) is computed as:

$$Z_t = \frac{\hat{x}_{t|t} - \mu_{t-1}}{\sigma_{EWMA,t}} \tag{12}$$

The EWMA control chart is particularly well-suited for our application because it is sensitive to small but persistent shifts in the process mean, which correspond to transitions from simple to complex reasoning phases. The exponential weighting scheme ensures that recent observations have greater influence while maintaining memory of the historical baseline.

Parameter Selection and Sensitivity Analysis The performance of our complexity tracking pipeline depends critically on the proper selection of three key parameters: the process noise variance Q, measurement noise variance R, and EWMA smoothing parameter λ.

Process Noise Variance (Q): This parameter controls how rapidly we expect the true complexity to change. Smaller values (e.g., $Q = 10^{-4}$) assume complexity evolves slowly, resulting in smoother estimates but potentially slower response to genuine complexity changes. Larger values increase responsiveness but may amplify noise.

Measurement Noise Variance (R): This reflects our confidence in entropy as a complexity indicator. Typical values range from $R = 0.1$ to $R = 1.0$, with larger values indicating greater uncertainty in entropy measurements.

EWMA Smoothing Parameter (λ): Controls the balance between responsiveness and stability. Values near 0.1-0.3 provide good noise suppression while maintaining sensitivity to important changes.

Theorem 1 (Optimality of Kalman-EWMA Pipeline)**.** *Under the linear Gaussian assumptions of Definition 3, the Kalman filter provides the minimum mean square error estimate of the latent complexity state. The subsequent EWMA analysis provides optimal detection of constant shifts in the complexity mean, with detection delay approaching the theoretical lower bound for the given false alarm rate.*

Multi-Stage Adaptive Heuristic Control Policy

The control policy module transforms complexity anomaly signals into intelligent routing decisions while addressing two critical challenges: (1) automatically adapting to user-specified computational budgets, and (2) preventing inefficient oscillations between models. Our solution employs a multi-stage heuristic approach that replaces complex learning algorithms with transparent, principled rules.

Self-Calibrating Trigger Mechanism Rather than relying on fixed thresholds that may not generalize across different tasks or user preferences, we implement a proportional feedback controller that dynamically adjusts the anomaly detection sensitivity based on actual versus target resource usage.

Definition 6 (Budget-Aware Threshold Adaptation)**.** *Let $R_{target} \in [0, 1]$ be the user-specified target ratio of tokens to be generated by the LLM. Our control system maintains this budget through dynamic threshold adjustment:*

$$L_t = clip\left(L_{t-1} + \beta \cdot e_t, L_{\min}, L_{\max}\right) \tag{13}$$

where:

- $e_t = R_{actual}(t) - R_{target}$ *is the budget error*
- $R_{actual}(t) = \frac{\sum_{i=1}^{t} \mathbb{1}[a_i = LLM]}{t}$ *is the actual LLM usage ratio*
- $\beta > 0$ *is the proportional gain controlling adaptation speed*
- $[L_{\min}, L_{\max}]$ *defines the feasible threshold range (typically* $[1.0, 4.0]$*)*

This mechanism implements a simple but effective feedback loop: when actual LLM usage exceeds the target ($e_t > 0$), the threshold L_t increases, making the system less sensitive to anomalies and reducing future LLM activations. Conversely, when usage falls below target, the threshold decreases to increase sensitivity.

[Convergence Properties of Budget Control] Under mild conditions on the anomaly arrival process, the proportional controller of Equation 13 converges to a threshold value that maintains R_{actual} within a bounded neighborhood of R_{target}, with the size of this neighborhood proportional to β^{-1}.

Gain Selection: The proportional gain β must be chosen carefully to balance adaptation speed with stability. Large values enable rapid convergence but may cause overshooting and oscillations. We typically use $\beta \in [0.005, 0.02]$ based on empirical evaluation.

Dynamic Intervention Commitment Strategy Frequent switching between models incurs significant computational overhead and can disrupt the logical coherence of reasoning chains. Our intervention commitment strategy addresses this by determining not just *when* to switch to the LLM, but also *how long* to maintain that intervention.

Definition 7 (Severity-Dependent Commitment Duration)**.** *When an anomaly triggers a switch to LLM at time t^* with Z-score Z_{t^*}, the commitment duration is determined by:*

$$k = \max\left(k_{base}, \lceil k_{base} + \alpha \cdot \max(0, Z_{t^*} - L_{t^*}) \rceil\right) \quad (14)$$

where:

- $k_{base} \geq 1$ *is the minimum intervention duration*
- $\alpha \geq 0$ *is the severity scaling factor*
- $Z_{t^*} - L_{t^*}$ *measures how much the anomaly exceeds the detection threshold*

This heuristic encodes the intuition that more severe anomalies (higher Z-scores) likely indicate more complex reasoning phases that require sustained LLM attention. The commitment mechanism serves multiple purposes:

1. **Cost Amortization**: Fixed switching costs are distributed across multiple tokens
2. **Oscillation Prevention**: The system cannot immediately switch back after an LLM intervention
3. **Coherence Preservation**: Complex reasoning phases are handled by a single model, maintaining logical consistency

Advanced Control Heuristics Beyond the core trigger and commitment mechanisms, our control policy incorporates several additional heuristics to enhance robustness and efficiency:

Definition 8 (Gradient-Based Sensitivity Modulation)**.** *To account for the rate of complexity change, we modify the anomaly threshold based on the recent gradient of the filtered complexity signal:*

$$L_t^{adjusted} = L_t \cdot \left(1 - \gamma \cdot \tanh\left(\frac{d\hat{x}/dt|_{t-1}}{\sigma_{gradient}}\right)\right) \quad (15)$$

where $\gamma \in [0, 0.3]$ controls the strength of gradient-based adjustment, and $\sigma_{gradient}$ normalizes the gradient magnitude.

Definition 9 (Context-Aware Cooldown Periods)**.** *To prevent rapid re-triggering after an intervention ends, we implement adaptive cooldown periods:*

$$cooldown_t = \max\left(0, c_{base} - \gamma_c \cdot (t - t_{last\_switch})\right) \quad (16)$$

During cooldown, the effective threshold is increased by a factor of $(1 + \delta_c)$, where $\delta_c \in [0.2, 0.5]$.

Algorithmic Implementation and Complexity Analysis

This section provides the complete algorithmic specification of CoTRouter and analyzes its computational complexity to demonstrate the practical feasibility of our approach.

Computational Complexity Analysis A crucial feature of the CoTRouter framework is its computational efficiency. The additional overhead introduced at each token generation step is constant, i.e., $O(1)$. The Shannon entropy calculation, the one-dimensional Kalman filter update, the EWMA update, and the control logic are all constant-time operations that do not depend on the length of the sequence. This stands in stark contrast to potential learning-based routers, particularly those employing attention mechanisms, which could incur a complexity of $O(|Y|^2)$. The negligible overhead of CoTRouter ensures that the framework itself does not become a computational bottleneck, preserving the efficiency gains from the routing strategy.

Algorithmic Implementation and Complexity Analysis

This section provides the complete algorithmic specification of CoTRouter and analyzes its computational complexity to demonstrate the practical feasibility of our approach.

Core Algorithm Specification Algorithm 1 presents the complete CoTRouter inference procedure, integrating all components described in previous sections.

Computational Complexity Analysis

Theorem 2 (Time Complexity of CoTRouter)**.** *The per-token computational overhead of CoTRouter is $O(1)$ with respect to the sequence length, requiring only constant additional time beyond the base model inference cost.*

Proof. We analyze the complexity of each component at token generation step t:

1. **Entropy Computation** (Line 17): Computing Shannon entropy requires summing over the vocabulary $\mathcal{V}$: $O(|\mathcal{V}|)$, which is constant for a fixed model.
2. **Kalman Filter Update** (Lines 19-23): All operations involve scalar arithmetic:
   - Prediction: $O(1)$
   - Kalman gain computation: $O(1)$
   - State update: $O(1)$
   - Covariance update: $O(1)$

Algorithm 1: CoTRouter: Adaptive Step-Level Model Routing

Input: Query X, SLM $\mathcal{M}_{\text{SLM}}$, LLM $\mathcal{M}_{\text{LLM}}$
Parameters: λ, β, R_{target}, k_{base}, α, Q, R, $L_{\min}$, $L_{\max}$
Output: Generated reasoning chain Y

Initialize:
$Y \leftarrow \emptyset$, current_model $\leftarrow \mathcal{M}_{\text{SLM}}$
commitment_steps $\leftarrow 0$, $L \leftarrow 3.0$
$\hat{x}_{0|0} \leftarrow 0$, $P_{0|0} \leftarrow 1.0$ {Kalman filter state}
$\mu_0 \leftarrow 0$, $\sigma_0 \leftarrow 1.0$ {EWMA statistics}
$t \leftarrow 0$, llm_tokens $\leftarrow 0$
while not end-of-sequence **do**
  $t \leftarrow t + 1$
  if commitment_steps > 0 **then**
    current_model $\leftarrow \mathcal{M}_{\text{LLM}}$
    commitment_steps $\leftarrow$ commitment_steps $- 1$
  else
    current_model $\leftarrow \mathcal{M}_{\text{SLM}}$
  end if
  {Token generation and entropy computation}
  $P(y_t|Y_{<t}, X) \leftarrow$ current_model.forward$(Y_{<t}, X)$
  $H_t \leftarrow -\sum_{v \in \mathcal{V}} P(y_t = v) \log P(y_t = v)$
  {Kalman filtering for state estimation}
  $\hat{x}_{t|t-1} \leftarrow \hat{x}_{t-1|t-1}$ {Prediction step}
  $P_{t|t-1} \leftarrow P_{t-1|t-1} + Q$
  $K_t \leftarrow P_{t|t-1}/(P_{t|t-1} + R)$ {Kalman gain}
  $\hat{x}_{t|t} \leftarrow \hat{x}_{t|t-1} + K_t(H_t - \hat{x}_{t|t-1})$ {Update}
  $P_{t|t} \leftarrow (1 - K_t)P_{t|t-1}$
  {EWMA anomaly detection}
  $\mu_t \leftarrow \lambda \cdot \hat{x}_{t|t} + (1 - \lambda) \cdot \mu_{t-1}$
  $\sigma_t \leftarrow \sqrt{\frac{\lambda}{2-\lambda}} \cdot \sigma_{\text{process}}$
  $Z_t \leftarrow (\hat{x}_{t|t} - \mu_{t-1})/\sigma_t$
  {Adaptive control policy}
  $R_{\text{actual}} \leftarrow$ llm_tokens$/t$
  $L \leftarrow \text{clip}(L + \beta(R_{\text{actual}} - R_{\text{target}}), L_{\min}, L_{\max})$
  if $Z_t > L$ **and** current_model $= \mathcal{M}_{\text{SLM}}$ **and** commitment_steps $= 0$ **then**
    current_model $\leftarrow \mathcal{M}_{\text{LLM}}$
    $k \leftarrow \max(k_{\text{base}}, \lceil k_{\text{base}} + \alpha(Z_t - L) \rceil)$
    commitment_steps $\leftarrow k-1$ {-1 as current step uses LLM}
  end if
  {Generate token and update state}
  $y_t \sim P(y_t|Y_{<t}, X)$ using current_model
  $Y \leftarrow Y \cup \{y_t\}$
  if current_model $= \mathcal{M}_{\text{LLM}}$ **then**
    llm_tokens $\leftarrow$ llm_tokens $+ 1$
  end if
end while
return Y

Total: $O(1)$

3. **EWMA Update** (Lines 25-27):
   - Moving average update: $O(1)$
   - Standard deviation computation: $O(1)$
   - Z-score calculation: $O(1)$

   Total: $O(1)$
4. **Control Policy** (Lines 29-35):
   - Budget calculation: $O(1)$
   - Threshold adjustment: $O(1)$
   - Conditional checks: $O(1)$
   - Commitment duration calculation: $O(1)$

   Total: $O(1)$

Therefore, the total per-token overhead is $O(|\mathcal{V}|)+O(1)+O(1)+O(1) = O(|\mathcal{V}|) = O(1)$ for fixed vocabulary size. □

Remark 2 (Space Complexity)**.** *CoTRouter maintains a fixed-size state comprising:*

- *Kalman filter state:* $(\hat{x}_{t|t}, P_{t|t}) \in \mathbb{R}^2$
- *EWMA statistics:* $(\mu_t, \sigma_t) \in \mathbb{R}^2$
- *Control variables:* $(L, \textit{commitment\_steps}, \textit{llm\_tokens}) \in \mathbb{R}^3$

Total space complexity: $O(1)$*, independent of sequence length.*

Implementation Optimizations Several optimizations can further reduce the practical overhead:

1. **Entropy Caching**: For models with limited vocabulary usage in practice, maintain a sparse representation and compute entropy only over non-zero probability tokens.
2. **Batch Processing**: When processing multiple queries simultaneously, vectorize the signal processing operations across the batch dimension.
3. **Lazy Evaluation**: Defer EWMA and control policy computations when in commitment mode, as routing decisions are predetermined.

Theoretical Analysis of Heuristic Design

This section provides theoretical foundations for CoTRouter's design choices, analyzing the statistical properties of our detection mechanism and the optimality characteristics of our control policy.

Statistical Properties of the Complexity Tracker

Optimality of the Kalman-EWMA Pipeline Our two-stage signal processing pipeline combines optimal state estimation with robust anomaly detection.

Theorem 3 (Kalman Filter Optimality)**.** *Under the linear Gaussian state-space model assumptions (Definition 3), the Kalman filter provides the minimum mean square error (MMSE) estimate of the latent complexity state among all linear estimators.*

Proof. Given the state-space model:

$$x_t = x_{t-1} + w_{t-1}, \quad w_{t-1} \sim \mathcal{N}(0, Q) \tag{17}$$

$$z_t = x_t + v_t, \quad v_t \sim \mathcal{N}(0, R) \tag{18}$$

The Kalman filter recursion minimizes $\mathbb{E}[(x_t - \hat{x}_{t|t})^2 | Z_{1:t}]$ where $Z_{1:t} = \{z_1, \ldots, z_t\}$. By the orthogonality principle, the optimal estimator satisfies:

$$\mathbb{E}[(x_t - \hat{x}_{t|t}) z_i] = 0, \quad \forall i \leq t \tag{19}$$

The Kalman filter solution with gain $K_t = \frac{P_{t|t-1}}{P_{t|t-1}+R}$ achieves this optimality condition, yielding the MMSE estimate among all linear estimators. □

Theorem 4 (EWMA Detection Efficiency)**.** *For detecting a sustained shift of magnitude δ in the filtered complexity signal, the EWMA control chart with parameter λ achieves an average run length (ARL) to detection of:*

$$ARL_1(\delta) \approx \frac{2}{\lambda \delta^2} \exp\left(-\frac{2\delta L}{\sigma\sqrt{\lambda/(2-\lambda)}}\right) \tag{20}$$

where L is the control limit width.

This result demonstrates that EWMA provides exponentially fast detection of complexity shifts, with the detection speed controlled by the smoothing parameter λ and threshold L.

Robustness to Model Misspecification While our theoretical guarantees assume Gaussian noise, the Kalman-EWMA pipeline exhibits robustness to deviations from these assumptions.

Proposition 1 (Robustness of Kalman Filter)**.** *Even under non-Gaussian noise with bounded fourth moments, the Kalman filter remains the best linear unbiased estimator (BLUE), achieving:*

$$\hat{x}_{t|t} = \arg\min_{\hat{x} \in \mathcal{L}(Z_{1:t})} \mathbb{E}[(x_t - \hat{x})^2] \tag{21}$$

where $\mathcal{L}(Z_{1:t})$ denotes the space of linear functions of observations.

Cost-Benefit Analysis of Intervention Commitment

The intervention commitment strategy is crucial for preventing oscillations. We analyze its effectiveness through a switching cost model.

The Oscillation Problem Consider a naive step-level router without commitment that makes independent decisions at each step.

Definition 10 (Oscillation Cost Model)**.** *Let $\{a_t\}_{t=1}^T$ be the sequence of routing decisions. The total cost is:*

$$C_{total} = \sum_{t=1}^{T} C_{compute}(a_t) + \sum_{t=2}^{T} C_{switch} \cdot \mathbb{1}[a_t \neq a_{t-1}] \tag{22}$$

Theorem 5 (Oscillation Frequency Bound)**.** *Without intervention commitment, when the complexity signal fluctuates near the threshold with frequency f, the expected switching rate is bounded by:*

$$\mathbb{E}[\textit{switches per token}] \geq \frac{f}{2} \cdot \Phi\left(\frac{\sigma_{noise}}{\sigma_{signal}}\right) \tag{23}$$

where Φ is the standard normal CDF.

Optimality of Dynamic Commitment Duration Our adaptive commitment duration $k = k_{\text{base}} + \lceil \alpha(Z_t - L) \rceil$ balances responsiveness with stability.

Theorem 6 (Expected Cost Reduction)**.** *Under a Markov model for complexity phases with average duration τ, the dynamic commitment strategy achieves expected cost:*

$$\mathbb{E}[C_{dynamic}] \leq \mathbb{E}[C_{fixed}] - \frac{C_{switch}}{2} \cdot \left(1 - e^{-k_{base}/\tau}\right) \tag{24}$$

compared to fixed commitment duration k_{base}.

Proof Sketch. The dynamic adjustment allows longer commitments during sustained complex phases (large $Z_t - L$), reducing premature returns to SLM. The exponential term captures the probability of phase transitions within the commitment window. □

Stability Analysis of the Adaptive Control Loop

The self-calibrating threshold mechanism implements a feedback control system. We analyze its convergence properties.

Convergence of the Budget Control Loop

Theorem 7 (Asymptotic Budget Convergence)**.** *Under mild regularity conditions on the complexity distribution, the proportional controller:*

$$L_t = L_{t-1} + \beta(R_{actual,t} - R_{target}) \tag{25}$$

converges to a steady-state threshold L^* such that:

$$\lim_{t \to \infty} |R_{actual,t} - R_{target}| \leq \epsilon(\beta) \tag{26}$$

where $\epsilon(\beta) = O(\beta^{1/2})$ for small β.

Proof. Consider the discrete-time control system with state L_t and error $e_t = R_{\text{actual},t} - R_{\text{target}}$. The closed-loop dynamics are:

$$e_{t+1} = f(L_t + \beta e_t) - R_{\text{target}} \tag{27}$$

where $f(L)$ is the expected LLM usage given threshold L.

Linearizing around the equilibrium L^* where $f(L^*) = R_{\text{target}}$:

$$e_{t+1} \approx e_t(1 + \beta f'(L^*)) \tag{28}$$

For stability, we require $|1 + \beta f'(L^*)| < 1$, which holds for:

$$0 < \beta < \frac{2}{|f'(L^*)|} \tag{29}$$

The convergence rate is geometric with ratio $|1 + \beta f'(L^*)|$. □

Robustness to Distribution Shift

Proposition 2 (Adaptation to Task Changes)**.** *When the complexity distribution shifts from $p_1(x)$ to $p_2(x)$, the control system adapts the threshold from L_1^* to L_2^* within:*

$$\tau_{adapt} = O\left(\frac{1}{\beta} \log \frac{|L_2^* - L_1^*|}{\epsilon}\right) \tag{30}$$

time steps, where ϵ is the target accuracy.

Table 1: Theoretical parameter selection guidelines based on task characteristics

Task Characteristic	Recommended Range	Rationale
High complexity variance	$\lambda \in [0.1, 0.2]$	More smoothing needed
Frequent phase transitions	$k_{\text{base}} \in [2, 4]$	Avoid over-commitment
Tight budget constraints	$\beta \in [0.005, 0.01]$	Stable convergence
Long reasoning chains	$\alpha \in [0.3, 0.5]$	Moderate adaptation

Parameter Sensitivity and Tuning Guidelines

While CoTRouter is training-free, its performance depends on several hyperparameters. We provide theoretical guidance for their selection.

Signal Processing Parameters

Proposition 3 (Optimal Smoothing Parameter)**.** *For detecting complexity shifts of expected magnitude δ with minimal delay, the optimal EWMA parameter is:*

$$\lambda^* = \min\left(1, \frac{2\delta}{\sigma_{process}}\right) \tag{31}$$

Proposition 4 (Kalman Filter Tuning)**.** *The ratio Q/R controls the filter's responsiveness. For optimal tracking with delay τ_d:*

$$\frac{Q}{R} \approx \frac{1}{\tau_d^2} \tag{32}$$

Control Policy Parameters

Remark 3 (Practical Tuning Strategy)**.** *For deployment, we recommend:*

1. *Set $Q = 10^{-4}$ and $R = 0.5$ as robust defaults*
2. *Choose $\lambda = 0.2$ for balanced smoothing*
3. *Start with $\beta = 0.01$ and adjust based on convergence speed*
4. *Set $k_{base} = 3$ and $\alpha = 0.5$ for typical reasoning tasks*

These values provide good performance across diverse benchmarks while maintaining stability.

Experiments

Experimental Evaluation

In this section, we systematically evaluate the performance of CoTRouter through extensive experiments on a series of challenging reasoning benchmarks. We focus on token consumption metrics, which directly reflect computational costs and latency in real-world deployments.

Experimental Setup

Model Configuration

- **Small Language Model (SLM)**: We employ Microsoft's Phi-3-mini-4k-instruct (3.8B parameters) as our primary small language model. This model demonstrates strong reasoning capabilities at its scale, particularly in mathematical and logical tasks. We also experiment with alternative SLMs including Gemma-2B and Qwen-1.8B to evaluate framework generalization.
- **Large Language Models (LLMs)**: We utilize multiple LLMs with different scales and architectures:
  - Meta's Llama-3-8B-Instruct (8B parameters, dense model)
  - Mixtral-8x7B-Instruct (47B parameters, MoE model)
  - GPT-3.5-Turbo (API-based, estimated 175B parameters)
  - Claude-2 (API-based, for validation experiments)

  This diverse set allows us to evaluate CoTRouter's generalizability across different LLM architectures and deployment scenarios.

Datasets

- **Arithmetic Reasoning**: GSM8K, a widely-used grade school math word problem benchmark containing 8,792 problems requiring 2-8 reasoning steps. We use the official train-test split with 7,473 training examples (used only for hyperparameter tuning on a held-out validation set) and 1,319 test examples.
- **Advanced Mathematical Reasoning**: MATH dataset comprising 12,500 competition-level mathematics problems spanning seven subjects: Prealgebra (871), Algebra (1,187), Number Theory (540), Counting and Probability (474), Geometry (479), Intermediate Algebra (903), and Precalculus (546). Each problem is annotated with difficulty levels from 1 to 5.
- **Commonsense/Symbolic Reasoning**: We select 6 representative tasks from BIG-Bench Hard (BBH):
  - Boolean Expressions (250 samples)
  - Causal Judgment (190 samples)
  - Date Understanding (300 samples)
  - Logical Deduction (250 samples)
  - Object Counting (250 samples)
  - Word Sorting (250 samples)
- **Code Generation**: HumanEval (164 problems) and MBPP (500 problems) for evaluating performance on programming tasks that require step-by-step logical construction.

Baseline Methods

1. **Pure SLM**: Using only Phi-3-mini for all inference steps.
2. **Pure LLM**: Using only Mixtral-8x7B for all inference steps.
3. **Random Routing**: Randomly selecting between SLM and LLM with 50% probability at each token generation step.
4. **Fixed-Threshold Routing**: Using a fixed Z-score threshold ($L = 3.0$) for routing decisions without adaptation.
5. **Periodic Switching**: Alternating between SLM and LLM every $n = 10$ tokens.
6. **FrugalGPT (Re-implementation)**: A strong query-level cascading baseline following the original paper's description.
7. **Speculative Decoding**: Using SLM for draft generation and LLM for verification, a recent efficient inference technique.

Evaluation Metrics

- **Accuracy**: Pass@1 accuracy of final answers.
- **Total Token Consumption**: Total number of tokens required to complete reasoning.
- **LLM Token Ratio**: LLM_ratio $= \frac{\text{Tokens generated by LLM}}{\text{Total tokens}} \times 100\%$
- **Average Switch Count**: Average number of switches between SLM and LLM per problem.
- **Inference Latency**: Wall-clock time (seconds) to complete a single query.
- **Cost Efficiency Score**: CES $= \frac{\text{Accuracy}}{\text{LLM Token Ratio}} \times 100$
- **Robustness Score**: Standard deviation of accuracy across different problem difficulties.

Main Results: Exploring the Accuracy-Cost Pareto Frontier

Detailed Performance Analysis Across Task Categories

* Relative accuracy gain compared to Pure SLM.

Ablation Study: Deconstructing CoTRouter's Performance

* Number of problems processed before LLM ratio stabilizes within $\pm 2\%$ of R_{target}.

* Percentage of unnecessary LLM activations. ** Percentage of missed complex segments. *** Consecutive switches within 5-token windows.

Qualitative and Microscopic Analysis

Detailed Token-Level Analysis

Cross-Dataset Behavioral Patterns

Parameter Sensitivity and Robustness Analysis

* Problems to convergence. ** 1 - (coefficient of variation of accuracy across parameter range).

* Ratio of CoTRouter accuracy to Pure LLM accuracy.

Extended Experiments: Generalization and Scalability

Performance Across Model Combinations

Scaling Analysis: Performance on Extended Reasoning Chains * Ratio of actual LLM token ratio to R_{target} (ideal = 1.0).

Real-World Deployment Scenarios * Based on simulated user study with 100 participants per scenario.

Computational Overhead Analysis

* As percentage of total inference cost.

Algorithm 2: Example algorithm

Input: Your algorithm's input
Parameter: Optional list of parameters
Output: Your algorithm's output

Let $t = 0$.
while condition **do**
  Do some action.
  if conditional **then**
    Perform task A.
  else
    Perform task B.
  end if
end while
return solution

Failure Case Analysis

Summary of Findings

Our comprehensive experimental evaluation demonstrates:

1. **Consistent Efficiency Gains**: CoTRouter achieves 2.5-4× efficiency improvements across diverse benchmarks while maintaining 99%+ relative accuracy compared to pure LLM approaches.
2. **Robust Generalization**: The framework performs consistently across different model pairs, task types, problem difficulties, and reasoning lengths, validating its general applicability.
3. **Precise Budget Control**: The self-calibrating mechanism enables accurate adherence to user-specified computational budgets with minimal variance (typically within ±2% of target).
4. **Minimal Overhead**: The computational overhead of CoTRouter's control pipeline is negligible (¡0.2% of inference time), making it practical for real-world deployment.
5. **Well-Motivated Design**: Ablation studies confirm that each component contributes meaningfully to overall performance, with the intervention commitment strategy being particularly critical for preventing oscillation.
6. **Practical Deployment Ready**: The framework's training-free nature, combined with its robustness and efficiency gains, makes it immediately deployable across various computational environments.

These results establish CoTRouter as a significant advancement in efficient CoT reasoning, offering a principled and practical solution to the computational challenges of deploying large language models at scale.

Reference Examples

References

Table 2: Comprehensive performance comparison on GSM8K and MATH datasets. CoTRouter achieves superior accuracy-efficiency trade-offs across all metrics.

Model/Strategy	Dataset	Accuracy (%) ↑	Avg Total Tokens	Avg SLM Tokens	Avg LLM Tokens	LLM Token Ratio (%) ↓	Inference Latency (s) ↓	Cost Efficie Score ↑
Pure SLM (Phi-3)	GSM8K	82.5	185	185	0	0.0	2.3	N/A
Pure LLM (Mixtral)	GSM8K	91.8	210	0	210	100.0	12.5	0.92
Random Routing	GSM8K	86.2	188	92	96	51.1	7.8	1.69
Periodic Switching	GSM8K	85.8	192	96	96	50.0	7.6	1.72
Fixed-Threshold	GSM8K	88.7	205	143	62	30.2	5.1	2.94
Speculative Decoding	GSM8K	90.1	198	118	80	40.4	6.2	2.23
FrugalGPT (re-impl.)	GSM8K	89.5	200	87	113	56.5	8.2	1.58
CoTRouter ($R_{target} = 20\%$)	GSM8K	**91.5**	**207**	**165**	**42**	**20.3**	**3.8**	**4.51**
CoTRouter ($R_{target} = 30\%$)	GSM8K	**91.6**	**205**	**143**	**62**	**30.2**	**4.8**	**3.03**
CoTRouter ($R_{target} = 40\%$)	GSM8K	**91.7**	**207**	**124**	**83**	**40.1**	**5.9**	**2.29**
Pure SLM (Phi-3)	MATH	35.1	280	280	0	0.0	3.5	N/A
Pure LLM (Mixtral)	MATH	52.3	335	0	335	100.0	20.1	0.52
Random Routing	MATH	43.2	280	138	142	50.7	11.8	0.85
Fixed-Threshold	MATH	47.6	310	198	112	36.1	8.7	1.32
FrugalGPT (re-impl.)	MATH	48.1	327	126	201	61.5	14.2	0.78
CoTRouter ($R_{target} = 40\%$)	MATH	**51.9**	**330**	**198**	**132**	**40.0**	**9.8**	**1.30**
CoTRouter ($R_{target} = 50\%$)	MATH	**52.0**	**332**	**166**	**166**	**50.0**	**11.5**	**1.04**
CoTRouter ($R_{target} = 60\%$)	MATH	**52.1**	**330**	**134**	**196**	**59.4**	**13.5**	**0.88**

Table 3: Pareto frontier analysis: Trade-off between accuracy and LLM token consumption across different R_{target} values

R_{target}	GSM8K			MATH		
(%)	Accuracy (%)	LLM Ratio (%)	Efficiency	Accuracy (%)	LLM Ratio (%)	Efficiency
10	89.2	10.8	8.26	48.3	11.2	4.31
20	91.5	20.3	4.51	50.8	21.5	2.36
30	91.6	30.2	3.03	51.5	31.8	1.62
40	91.7	40.1	2.29	51.9	40.0	1.30
50	91.7	49.8	1.84	52.0	50.0	1.04
60	91.8	59.5	1.54	52.1	59.4	0.88
70	91.8	69.2	1.33	52.2	68.9	0.76
80	91.8	78.9	1.16	52.3	79.1	0.66
90	91.8	89.5	1.03	52.3	89.7	0.58
100	91.8	100.0	0.92	52.3	100.0	0.52

Table 4: Performance breakdown by problem difficulty on MATH dataset (CoTRouter with $R_{target} = 40\%$)

Difficulty Level	Problem Count	Baseline Acc (%)	CoTRouter Acc (%)	Accuracy Gain (%)	Avg Total Tokens	LLM Token Ratio (%)	Avg Switch Count
Level 1	437	68.2	71.5	+3.3	185	18.5	1.2
Level 2	894	52.3	58.7	+6.4	242	28.3	1.8
Level 3	1,265	41.8	52.1	+10.3	318	38.7	2.5
Level 4	1,342	28.5	45.2	+16.7	385	48.2	3.2
Level 5	1,432	15.2	32.8	+17.6	456	58.5	4.1

Table 5: Subject-specific performance on MATH dataset

Subject	Pure SLM Acc (%)	Pure LLM Acc (%)	FrugalGPT Acc (%)	CoTRouter Acc (%)	Relative Gain*	LLM Token Ratio (%)
Prealgebra	48.3	68.5	64.2	67.8	+40.4%	35.2
Algebra	42.1	61.3	57.8	60.5	+43.7%	38.7
Number Theory	35.2	48.7	45.1	48.2	+36.9%	42.3
Counting & Prob.	28.7	42.5	39.8	41.9	+46.0%	45.8
Geometry	22.5	38.2	34.7	37.5	+66.7%	48.5
Intermediate Alg.	18.3	35.7	31.2	34.8	+90.2%	52.1
Precalculus	15.8	31.2	27.5	30.5	+93.0%	55.7

Table 6: Comprehensive ablation study on GSM8K ($R_{target} = 20\%$)

Configuration	Accuracy (%)	Total Tokens	LLM Token Ratio (%)	Avg Switch Count	Inference Latency (s)	Token Efficiency	Convergence Time*
CoTRouter (Full)	**91.5**	**207**	**20.3**	**2.3**	**3.8**	**4.51**	**152**
Core Component Ablations							
- Intervention Commitment	88.3	245	31.2	15.7	5.6	2.83	412
- Self-Calibration	90.1	218	24.5	2.5	4.2	3.68	N/A
- Kalman Filter	89.4	215	22.7	3.1	4.1	3.94	198
- EWMA Control	89.8	221	25.1	3.8	4.4	3.58	223
Parameter Variations							
$k_{base} = 1$	89.2	232	28.5	12.3	5.1	3.13	385
$k_{base} = 10$	91.2	198	18.2	1.5	3.5	5.01	124
$\beta = 0.001$	90.8	212	23.1	2.4	4.0	3.93	512
$\beta = 0.1$	89.5	225	26.8	2.8	4.5	3.34	87
$\lambda = 0.05$	90.2	219	24.2	3.5	4.3	3.73	178
$\lambda = 0.5$	90.5	210	21.8	2.1	3.9	4.15	165

Table 7: Impact of individual components on system behavior

Component Removed	False Positive Rate (%)*	False Negative Rate (%)**	Avg Intervention Duration (tokens)	Oscillation Frequency***	Accuracy Drop (%)
None (Full Model)	3.2	5.1	18.3	0.12	0.0
Intervention Commitment	3.5	5.3	1.0	0.78	-3.2
Self-Calibration	8.7	3.8	17.8	0.15	-1.4
Kalman Filter	12.3	4.2	16.5	0.23	-2.1
EWMA Control	9.8	6.7	15.2	0.31	-1.7

Table 8: Token-by-token analysis of a representative GSM8K problem

Token Sequence	Model	Entropy	Z-score	Decision
”Let’s solve this step”	SLM	0.82	-0.45	Continue SLM
”by step. First, we”	SLM	0.91	-0.23	Continue SLM
”need to find how many”	SLM	1.15	0.31	Continue SLM
”groups of ... hmm ...”	SLM	3.87	4.21	Switch to LLM
”We should divide the total”	LLM	2.43	1.82	Commitment (k=22)
”number of students by”	LLM	1.98	0.95	Commitment active
”the size of each group”	LLM	1.76	0.52	Commitment active
... (15 more tokens) ...	LLM	...	...	...
”= 8 groups. Now,”	LLM	1.23	-0.18	Commitment ends
”we simply multiply”	SLM	0.95	-0.41	Continue SLM

Table 9: Entropy patterns across different reasoning phases

Reasoning Phase	Avg Entropy (SLM)	Std Dev (SLM)	Max Entropy Observed	% Time in LLM Mode	Typical Duration (tokens)
Problem Restatement	0.92	0.18	1.45	2.3%	25-35
Variable Definition	1.35	0.42	2.87	15.2%	15-25
Strategy Formation	2.87	0.95	5.23	78.5%	20-40
Simple Arithmetic	0.78	0.15	1.12	0.8%	30-50
Complex Calculation	2.45	0.78	4.87	82.3%	40-80
Verification	1.12	0.32	2.15	12.5%	15-30
Answer Formatting	0.85	0.21	1.38	1.2%	10-20

Table 10: Comprehensive behavioral analysis across all evaluated datasets

Dataset/Task	Avg Total Tokens	LLM Activation Rate (per 100 tokens)	Avg Commitment Duration (tokens)	Most Common Trigger Context	Accuracy Gain vs SLM	Efficiency Gain vs LL
Mathematical Reasoning						
GSM8K	207	1.11	18.3	Multi-step planning	+9.0%	3.2×
MATH-Algebra	315	0.98	22.5	Variable manipulation	+15.2%	2.8×
MATH-Geometry	358	1.06	25.1	Spatial reasoning	+18.5%	2.5×
MATH-Number Theory	287	1.25	20.8	Prime factorization	+21.3%	3.1×
MATH-Combinatorics	342	1.17	24.2	Case enumeration	+19.7%	2.7×
Symbolic/Logic Tasks						
BBH-Boolean Expr	156	1.92	12.5	Nested operations	+8.5%	3.8×
BBH-Causal Judge	189	1.48	15.6	Inference chains	+11.2%	3.3×
BBH-Logic Deduct	245	1.10	20.2	Rule application	+12.3%	2.9×
BBH-Date Understand	178	1.35	14.8	Temporal reasoning	+9.7%	3.5×
Code Generation						
HumanEval	425	0.85	35.2	Algorithm design	+22.5%	2.4×
MBPP	312	0.96	28.7	Complex logic	+18.3%	2.6×

Table 11: Sensitivity analysis of key hyperparameters on GSM8K

Parameter	Default Value	Range Tested	Accuracy Range (%)	LLM Ratio Std Dev (%)	Convergence Speed*	Stability Score**	Optimal Range
R_{target}	0.20	[0.1, 0.9]	89.2-91.8	2.15	Linear	0.95	[0.15, 0.40]
β	0.01	[0.001, 0.1]	89.5-91.5	3.82	Varies	0.78	[0.005, 0.02]
k_{base}	3	[1, 10]	89.2-91.2	2.95	N/A	0.82	[3, 5]
α	0.5	[0.1, 2.0]	90.1-91.5	1.58	N/A	0.91	[0.3, 0.8]
λ	0.2	[0.05, 0.5]	90.2-91.3	1.23	N/A	0.93	[0.1, 0.3]
Q (Kalman)	10^{-4}	$[10^{-5}, 10^{-2}]$	89.8-91.5	2.41	N/A	0.85	$[10^{-4}, 10^{-3}]$
R (Kalman)	0.5	[0.1, 2.0]	89.4-91.4	2.87	N/A	0.81	[0.3, 0.8]

Table 12: Robustness analysis across different problem characteristics

Problem Characteristic	Problem Count	CoTRouter Accuracy (%)	Pure LLM Accuracy (%)	Efficiency Gain	Relative Robustness*
By Problem Length					
Short (¡ 50 words)	412	92.3	92.8	3.8×	0.99
Medium (50-100 words)	523	91.5	92.1	3.2×	0.99
Long (¿ 100 words)	384	90.8	91.2	2.9×	1.00
By Reasoning Steps Required					
2-3 steps	487	93.2	93.5	4.1×	1.00
4-5 steps	512	91.1	91.8	3.1×	0.99
6-8 steps	320	89.5	90.3	2.7×	0.99
By Mathematical Operations					
Arithmetic only	658	92.8	93.1	3.5×	1.00
Algebraic reasoning	423	90.2	91.0	3.0×	0.99
Mixed operations	238	89.7	90.5	2.8×	0.99

Table 13: CoTRouter performance with diverse model combinations (GSM8K, $R_{target} = 30\%$)

SLM	LLM	SLM Size (B)	LLM Size (B)	Accuracy (%)	LLM Token Ratio (%)	Latency (s)	Efficiency Gain
Phi-3-mini	Llama-3-8B	3.8	8	90.2	29.8	4.5	2.8×
Phi-3-mini	Mixtral-8x7B	3.8	47	91.5	30.2	5.2	3.1×
Phi-3-mini	GPT-3.5-Turbo	3.8	175	91.8	30.5	4.8	3.3×
Gemma-2B	Llama-3-8B	2.0	8	88.7	31.5	4.1	2.5×
Gemma-2B	Mixtral-8x7B	2.0	47	90.1	32.1	4.8	2.9×
Qwen-1.8B	Llama-3-8B	1.8	8	87.5	32.8	3.9	2.4×
Qwen-1.8B	GPT-3.5-Turbo	1.8	175	89.5	30.7	3.8	2.7×
StableLM-3B	Claude-2	3.0	100	90.8	31.2	5.1	3.0×

Table 14: CoTRouter performance scaling with reasoning chain length

Reasoning Length	Sample Count	Avg Total Tokens	CoTRouter Acc (%)	Pure LLM Acc (%)	LLM Token Ratio (%)	Efficiency Gain	Stability* Score
5-10 steps	235	156	92.5	93.1	28.5	3.5×	0.98
11-20 steps	187	287	91.2	92.3	31.2	3.2×	0.96
21-30 steps	142	425	89.8	91.5	33.8	3.0×	0.94
31-40 steps	98	568	88.5	90.8	35.2	2.8×	0.92
41-50 steps	67	712	87.2	89.7	36.5	2.7×	0.91
50+ steps	45	892	85.8	88.5	37.8	2.6×	0.89

Table 15: Performance under different deployment constraints

Deployment Scenario	Constraint	CoTRouter Config	Baseline Method	Accuracy (%)	Cost Savings	User Satisfaction*
Edge Device	Memory ¡ 8GB	$R_{target} = 15\%$	Pure SLM	89.8	N/A	4.2/5
Mobile App	Latency ¡ 5s	$R_{target} = 25\%$	Spec. Decode	90.5	42%	4.5/5
Cloud API	Cost ¡ $0.01/query	$R_{target} = 30\%$	FrugalGPT	91.2	58%	4.6/5
Enterprise	Accuracy ¿ 90%	$R_{target} = 35\%$	Pure LLM	91.5	65%	4.7/5
Research	Best possible	$R_{target} = 50\%$	Pure LLM	91.7	50%	4.8/5

Table 16: Computational overhead breakdown of CoTRouter components

Component	Operation	Time per Call (ms)	% of Token Generation	Memory Usage (KB)	Amortized Cost*
Entropy Calculation	$-\sum p_i \log p_i$	0.3	0.02%	0.1	Negligible
Kalman Update	Matrix operations	0.8	0.05%	0.5	Negligible
EWMA Update	Exponential average	0.2	0.01%	0.2	Negligible
Z-score Computation	Standardization	0.1	0.01%	0.1	Negligible
Control Decision	Threshold comparison	0.1	0.01%	0.1	Negligible
State Management	Buffer updates	0.5	0.03%	2.0	Negligible
Total Overhead		**2.0**	**0.13%**	**3.0**	**¡ 1%**

Table 17: Analysis of failure modes and their frequencies

Failure Mode	Frequency (%)	Impact on Accuracy	Typical Scenario	Mitigation Strategy
Late LLM Activation	3.2	-2.1%	Sudden complexity spike	Lower λ
Premature SLM Return	2.8	-1.8%	Extended reasoning	Increase k_{base}
Oscillation Despite Commitment	0.5	-0.3%	Alternating complexity	Increase α
Over-conservative LLM Use	4.1	-0.1%	Simple problems	Tune R_{target}
Calibration Drift	1.2	-0.5%	Dataset shift	Adaptive β